

HW Week2: Day 5~7 Labs Report

김종원

Day5. LAB1 – RISC-V Program Counter

Program Counter(PC) 모듈은 내부 register인 `if_addr_r`를 이용하여 현재 명령어의 주소(Program Counter)를 저장한다. Program Counter는 clock의 positive edge마다 상태가 갱신되며, combinational logic과 달리 입력이 변한다고 즉시 주소가 변경되지 않고 clock edge에서만 새로운 주소가 저장된다.

Program Counter의 주요 입출력은 다음과 같다.

- `clk_i` : Program Counter 동작의 기준이 되는 clock 신호
- `reset_i` : Reset 신호로, 활성화되면 PC를 초기 주소(RESET_SP)로 초기화한다.
- `ird` : Instruction Read request 신호로, 명령어를 읽을 때 PC를 갱신한다.
- `branch_taken_w` : Branch 명령이 수행되는지를 나타내는 신호이다.
- `jump_addr_w` [31:0] : Branch가 발생했을 때 이동할 목적지 주소이다.
- `if_next_addr_w` [31:0] : 다음에 Memory로 전달되는 Program Counter 주소이다.

Program Counter는 Reset이 활성화되면 내부 register `if_addr_r`를 RESET_SP(0x00000000)으로 초기화한다. 이후 `ird=1`인 경우에만 Program Counter가 갱신된다.

Branch가 발생하지 않는 일반 명령어에서는 다음 명령어를 수행하기 위해 Program Counter가 4씩 증가한다. 이는 RV32I 명령어가 모두 32bit(4Byte) 크기를 가지므로 다음 명령어의 시작 주소가 항상 4Byte 뒤에 위치하기 때문이다.

반면 Branch 조건이 만족하여 `branch_taken_w=1`이 되면 Program Counter는 `jump_addr_w`에 저장된 주소로 변경된다. Program Counter는 Branch offset을 직접 계산하지 않으며, 이미 계산된 목적지 주소를 입력받아 해당 주소로 이동하는 역할만 수행한다.

TB1에 대한 시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 Reset 이후 Program Counter가 0x00000000으로 초기화되는 것을 확인할 수 있다. 이후 명령어를 읽을 때마다 Memory에는 다음과 같은 주소가 순차적으로 전달된다.

0000, 0004, 0008, 000C, 0010, 0014, 0018, 001C, 0020 ...

이는 Program Counter가 일반 명령어 수행 시마다 $PC \leftarrow PC + 4$ 로 정상적으로 증가함을 보여준다.

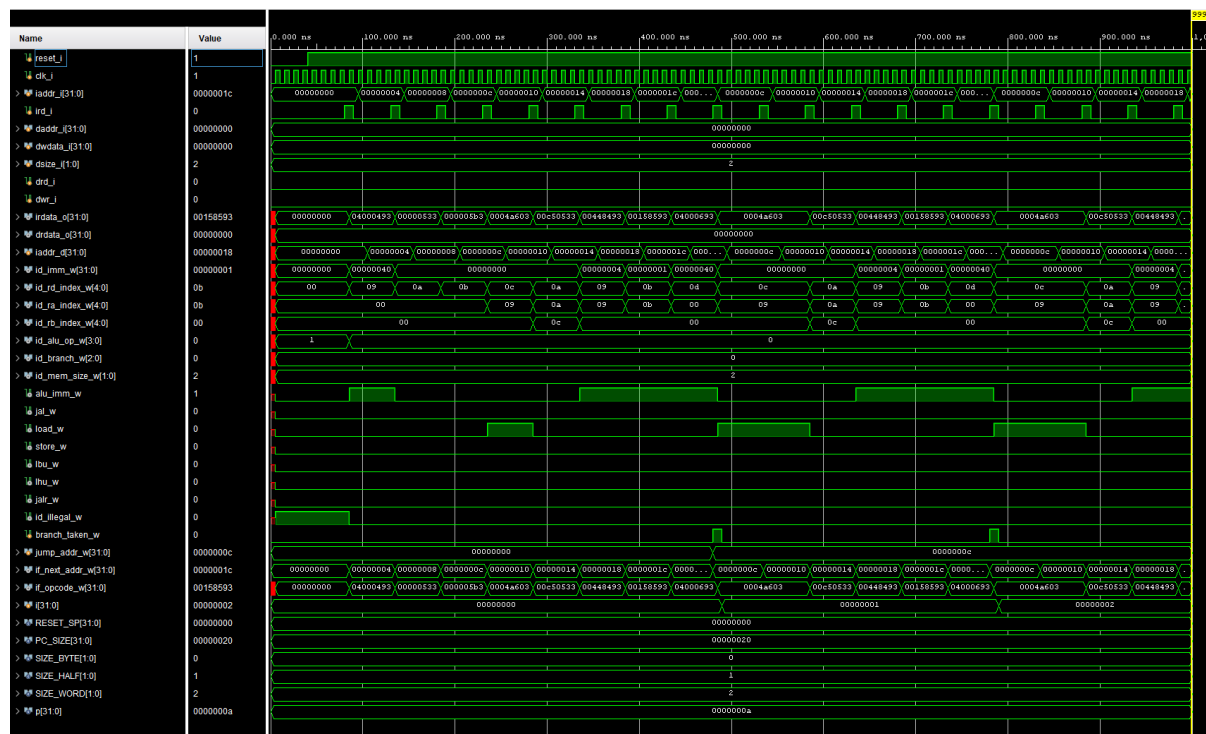
Waveform의 irdata_o에서는 각 주소에 저장된 RISC-V 명령어가 정상적으로 출력되는 것을 확인할 수 있으며, Decoder 역시 해당 명령어를 해석하여 id_rd_index_w, id_ra_index_w, id_rb_index_w, id_imm_w 등의 신호를 정상적으로 생성하는 것을 확인하였다.

또한, 두 가지 테스트 케이스를 통해 Program Counter의 동작을 확인하였다. 첫 번째 테스트에서는 Branch 없이 명령어를 순차적으로 실행하여 Program Counter가 0x0000 → 0x0004 → 0x0008 → 0x000C → 0x0010 → 0x0014 → 0x0018 → 0x001C → 0x0020 순으로 4Byte씩 증가하는 것을 확인하였다.

두 번째 테스트에서는 branch_taken_w = 1과 jump_addr_w = 0x0000000C를 입력하여 Branch 동작을 수행하였으며, Program Counter가 0x0020에서 0x000C로 정상적으로 이동한 후 다시 순차적으로 명령어를 수행하는 것을 확인하였다. 이를 통해 Program

Counter는 일반 명령어에서는 PC+4로 주소를 갱신하고, Branch가 발생하면 외부에서 전달된 목적지 주소로 이동하여 명령어의 실행 흐름을 변경한다는 것을 알게 되었다.

그리고, TB2에 대한 시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 Reset 이후 Program Counter가 0x00000000으로 초기화되는 것을 확인할 수 있다. TB2에서는 Memory의 입력 주소(iaddr_i)가 Program Counter의 출력(if_next_addr_w)과 직접 연결되어 있으므로, 즉, TB1에서는 Testbench가 Program Counter의 동작을 확인하기 위한 환경을 제공하였다면, TB2에서는 Program Counter가 실제로 Memory를 직접 제어하도록 구성되었다. 이에 따라 명령어 주소는 0x0000 → 0x0004 → 0x0008 → 0x000C → 0x0010 → 0x0014 → 0x0018 → 0x001C → 0x0020 순으로 증가하며, Program Counter가 정상적으로 $PC \leftarrow PC + 4$ 를 수행하는 것을 확인하였다.

또한, irdata_o에서는 각 주소에 저장된 RISC-V 명령어가 정상적으로 출력되었으며, Decoder 역시 명령어를 해석하여 id_rd_index_w, id_ra_index_w, id_rb_index_w, id_imm_w 등의 제어 신호를 정상적으로 생성하는 것을 확인하였다.

Branch 테스트에서는 branch_taken_w = 1과 jump_addr_w = 0x0000000C를 입력하였으며, Program Counter가 0x0020에서 0x0000000C로 정상적으로 분기된 이후 다시 0x0010 → 0x0014 → 0x0018 → 0x001C → 0x0020 순으로 명령어를 수행하는 것을 확인하였다.

이러한 과정들을 통해 Program Counter가 CPU의 명령어 실행 순서를 제어하며, 일반 명령어에서는 PC+4로 주소를 갱신하고 Branch가 발생하면 목적지 주소로 이동하여 명령어의 실행 흐름을 변경한다는 것을 알게 되었다.

Day5. LAB2 – Simplified RISC-V core

Baseline에서는 Program Counter가 매 Clock마다 명령어 주소를 생성하여 Memory로 전달하고, Decoder가 명령어를 해석하여 Register File과 ALU를 제어한다. 또한 Branch-Less-Than(BLT)의 기본 동작을 확인하기 위해 Branch Enable은 항상 1로 설정하고 Branch Target Address는 0x00000000으로 고정하였다. 따라서 BLT 명령어가 실행되면 조건과 관계없이 Program Counter는 항상 주소 0으로 분기한다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 Branch 동작에서 BLT 명령어가 실행되는 시점마다 branch_taken_w가 활성화되고, jump_addr_w가 기본값인 0x00000000으로 설정되어 Program Counter가 0x0020 → 0x0024 → 0x00000000으로 분기하는 것을 확인하였다. 이후 동일한 명령어를 다시 수행하면서 Program Counter는 0x0000 → 0x0004 → 0x0008 → ... → 0x0020 → 0x0024 → 0x0000의 실행 흐름을 반복하였다.

Branch 동작에서 Program Counter가 0x0020에서 바로 0x00000000으로 이동하지 않

고 0x00000024를 한 번 거친 후 분기하는 이유는 BLT 명령어가 먼저 Fetch된 이후 Decoder와 Branch Logic에서 분기 여부가 결정되기 때문이다. 따라서 Program Counter는 먼저 다음 순차 주소인 0x00000024를 생성한 뒤, Branch 제어 신호가 활성화되는 다음 Clock에서 목적지 주소인 0x00000000으로 변경된다.

Problem 2(b)에서 요구한 Register File의 r11(x11) 값 역시 확인하였다. 프로그램에서는 먼저 add x11, x0, x0 명령어에 의해 x11이 0으로 초기화되고, 이후 addi x11, x11, 1 명령어에 의해 x11이 1이 된다. 그러나 BLT에서는 Branch Enable이 항상 1이고 목적지 주소가 0x00000000으로 고정되어 있으므로, BLT 명령이 실행될 때마다 프로그램이 처음으로 되돌아가 다시 x11이 0으로 초기화된다. 따라서 x11은 0과 1 두 값만 반복적으로 저장되는 것을 확인하였다.

Problem 2(c)에서는 Branch-Less-Than(BLT) 명령어의 목적지 주소(Target Address) 계산과 Branch Enable 기능을 구현하였다. 이에 대한 시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 BLT 명령어가 Decode되는 시점에 jump_addr_w가 기본값인 0x00000000에서 분기 목적지 주소인 0x0000000C로 변경되는 것을 확인할 수 있다. 이는 Decoder에서 생성한 Immediate 값(id_imm_w)과 현재 Program Counter(if_pc_d)를 더하여 Branch 목적지 주소인 0x0000000C가 정상적으로 계산되는 것을 확인할 수 있다.

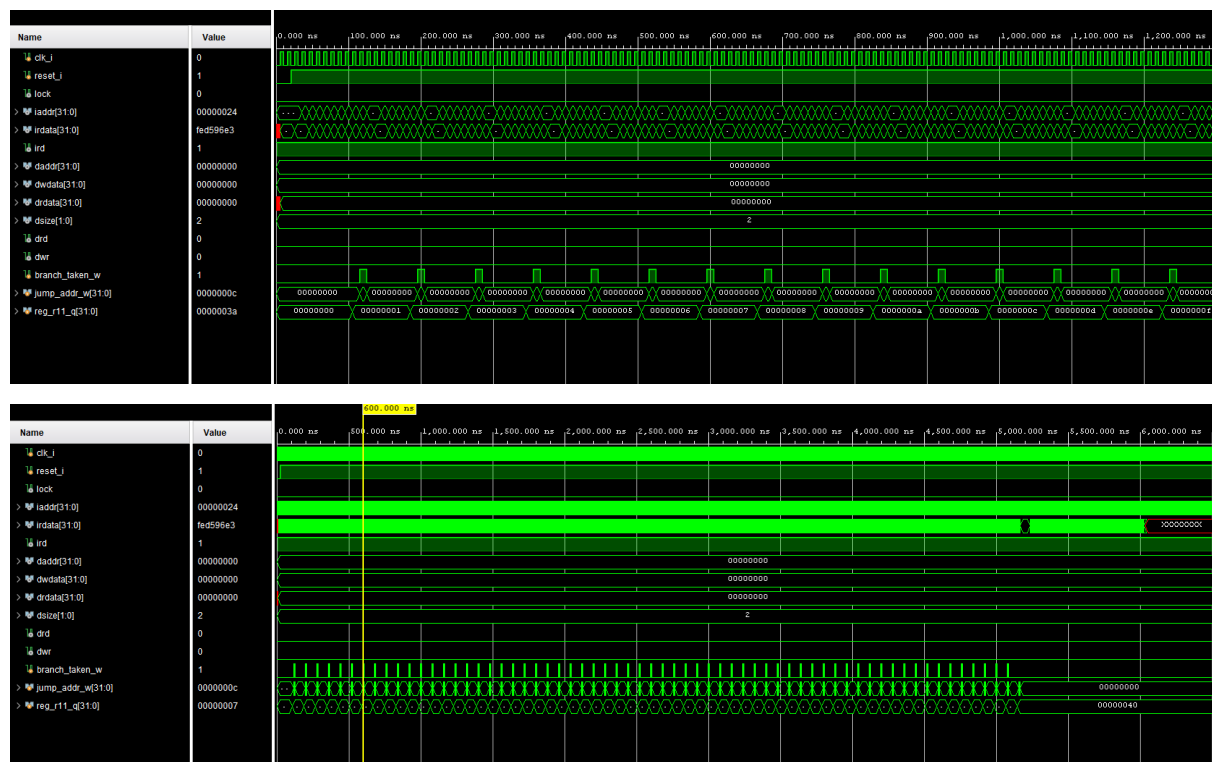
또한, branch_taken_w는 $ra_value_r < rb_value_r$ 조건을 만족하는 경우에만 활성화되었으며, jump_addr_w가 0x0000000C로 계산되는 시점과 동일한 Clock에서 활성화되는 것을 확인하였다. 이를 통해 BLT 명령어의 목적지 주소 계산과 Branch 조건 판단이 정상적

으로 수행됨을 확인하였다.

Problem 3에서는 Branch-Less-Than(BLT) 명령어를 Branch-Not-Equal(BNE) 명령어로 변경하여 동작을 확인하였다. 이를 위해 기존 mem.hex의 BLT 명령어를 BNE 명령어로 수정한 mem_bne.hex를 생성하였으며, Testbench 역시 새로운 Memory 파일을 사용하도록 riscv_core_sim_bne_tb.v로 수정하였다. 또한 BNE 명령어가 수행될 수 있도록 Branch Target Address 계산과 Branch Enable 생성 기능을 구현하였다.

BNE 명령어의 Branch Target Address는 BLT와 동일하게 현재 Program Counter와 Decoder에서 생성한 Immediate 값(id_imm_w)을 이용하여 계산하였다. Branch Enable은 두 Register의 값이 서로 다를 때 활성화되도록 구현하였다. 즉, $ra_value_r \neq rb_value_r$ 인 경우에만 Branch가 수행되며, 두 값이 같으면 다음 명령어를 순차적으로 실행하도록 하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 BNE 명령어가 Decode되는 시점마다 jump_addr_w가 Branch 목적지 주소인 0x0000000C로 계산되는 것을 확인할 수 있다. 또한 branch_taken_w는 Register File의 x11과 x13의 값이 서로 다른 동안에는 활성화되어 Program Counter가 0x00000020 → 0x00000024 → 0x0000000C의 실행 흐름을 반복하였다. 이는 Program Counter가 먼저 다음 순차 주소를 생성한 후, Branch 조건이 만족되면 다음 Clock에서 계산된 목적지 주소로 이동하기 때문이다.

Register File의 x11은 반복문이 수행될 때마다 1씩 증가하여 0x00 → 0x01 → 0x02 → ... → 0x3F → 0x40까지 증가하는 것을 확인하였다. x13에는 반복문의 종료 조건인 64(0x40)가 저장되어 있으므로, x11이 0x40이 되기 전까지는 $x11 \neq x13$ 조건이 만족되어 Branch가 계속 수행된다. 이후 x11과 x13이 모두 0x40이 되면 Branch 조건이 거짓이 되어 branch_taken_w가 비활성화되고, Program Counter는 더 이상 Loop의 시작 주소인 0x0000000C로 분기하지 않고 다음 명령어를 순차적으로 수행한다.

이를 통해 BLT와 BNE는 모두 동일한 Branch Target Address 계산 방식을 사용하지만, Branch 여부를 판단하는 조건만 서로 다르다는 것을 확인하였다. 또한 mem_bne.hex와 riscv_core_sim_bne_tb.v를 이용한 시뮬레이션을 통해 BNE 명령어의 Target Address 계산과 Branch Enable 생성 기능이 정상적으로 구현되었음을 알게 되었다.

Day6. LAB1 – Load, Store

(a) baseline의 시뮬레이션 결과는 아래 사진과 같다.



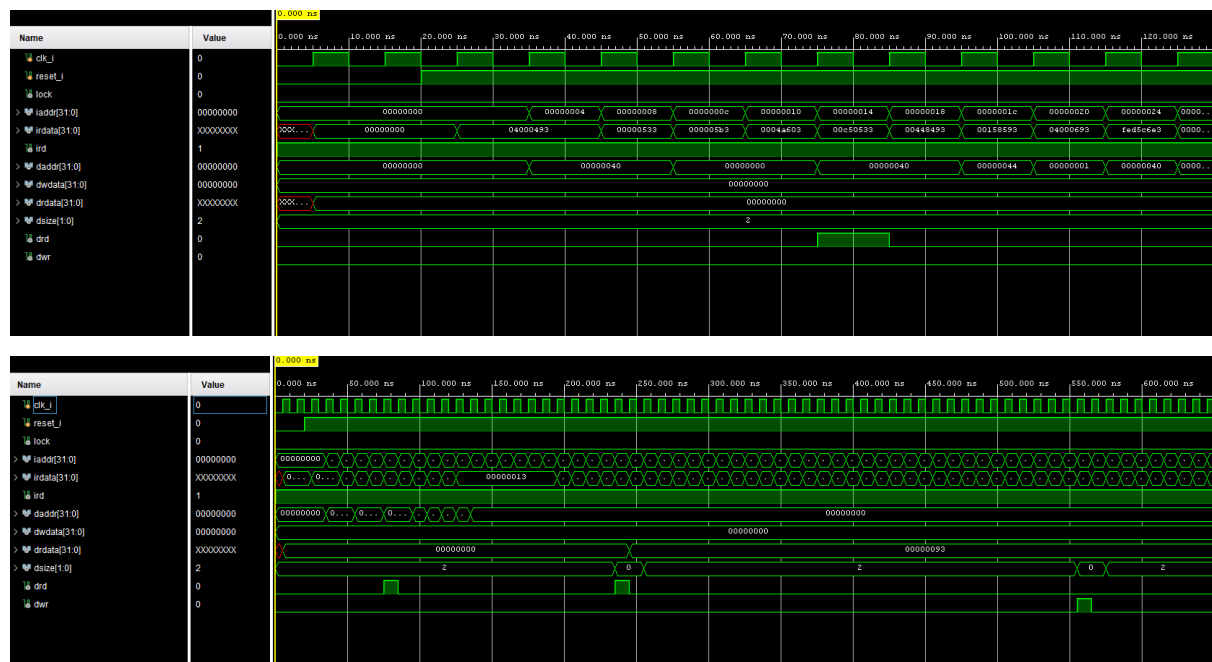
Problem 4(a)에서는 Program Counter를 완성하고, ALU, Decoder, Register File의 Dummy 코드를 활성화하여 Baseline RISC-V Core를 구성하였다. 또한 이전 과제에서 구현한 Branch-Less-Than(BLT) 기능을 적용한 후 시뮬레이션을 수행하였다. 시뮬레이션 결과는 아래 그림과 같다.

Waveform을 보면 Program Counter(iaddr)가 Reset 이후 0x00000000 → 0x00000004

→ 0x00000008 → 0x0000000C → 0x00000010 → 0x00000014 → 0x00000018 → 0x0000001C → 0x00000020 → 0x00000024 순으로 증가하는 것을 확인할 수 있다. 이는 Program Counter가 정상적으로 4Byte 단위로 명령어를 Fetch하고 있음을 의미한다. 또한 Instruction Memory로부터 전달되는 명령어(irdata)가 Program Counter의 주소 변화에 따라 순차적으로 출력되는 것을 확인하였다.

Memory Interface와 관련된 출력 신호인 daddr, dwdata, drd, dwr은 모두 비활성 상태를 유지하였다. 이는 Baseline 단계에서는 Load 및 Store 기능이 아직 구현되지 않았기 때문이며, 데이터 메모리에 대한 접근이 발생하지 않는 것이 정상적인 동작이다. 따라서 Instruction Fetch와 ALU 및 Register File의 연결이 정상적으로 수행되며, Baseline RISC-V Core가 올바르게 동작함을 확인하였다.

(b) Load, Store instructions의 시뮬레이션 결과는 아래 사진과 같다.



Problem 4(b)에서는 Baseline에서 사용한 Dummy 코드를 실제 Load 및 Store 기능으로 교체하였다. Execution Stage에서 계산된 ALU 결과를 데이터 메모리 주소로 전달하고, Store 명령어의 경우 Register File의 값을 메모리에 기록하도록 구현하였다. 또한 Load 명령어에서는 데이터 메모리로부터 읽어온 데이터를 Register File에 기록하도록 Write Back 경로를 수정하였다. 시뮬레이션 결과는 아래 그림과 같다.

Waveform을 보면 Load/Store 명령어가 수행되는 시점에 daddr 신호가 0x00000040 → 0x00000000 → 0x00000040 → 0x00000001 → 0x00000040과 같이 ALU에서 계산된 주소를 출력하는 것을 확인할 수 있다. 또한 dwdata에는 Store 명령어에서 메모리에 저장될 데이터가 출력되며, drd 신호는 Load 명령어 수행 시 활성화되고, dwr 신호는 Store

명령어 수행 시 활성화되는 것을 확인하였다.

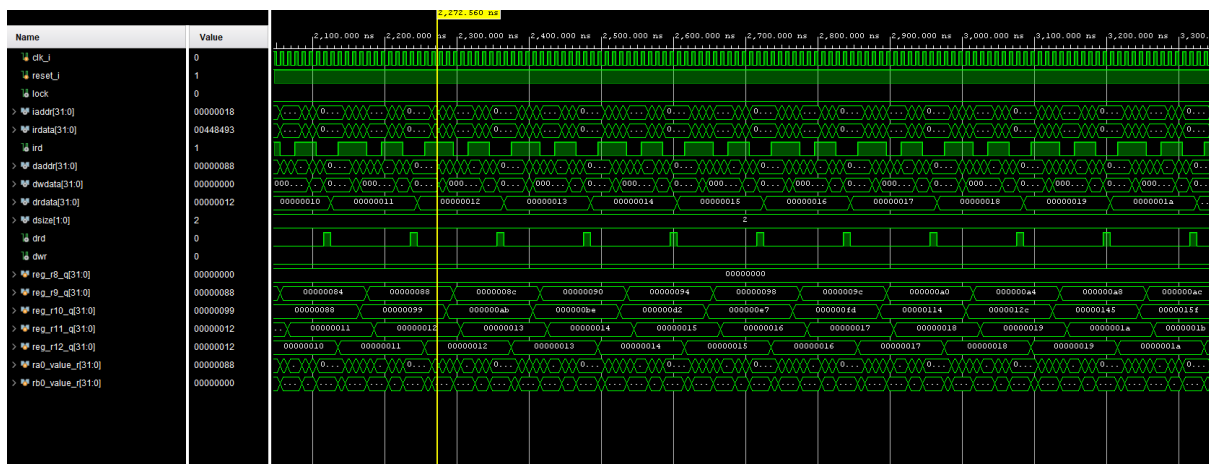
특히 약 70 ns 부근에서 dwr 신호가 활성화되며 Store 동작이 수행되는 것을 확인할 수 있다. 이때 데이터 메모리 주소와 저장할 데이터가 동시에 출력되어 메모리 쓰기 동작이 정상적으로 이루어짐을 확인하였다. 이후 Load 명령어에서는 메모리로부터 읽은 데이터가 Register File로 전달되어 정상적으로 Write Back되는 것을 확인하였다.

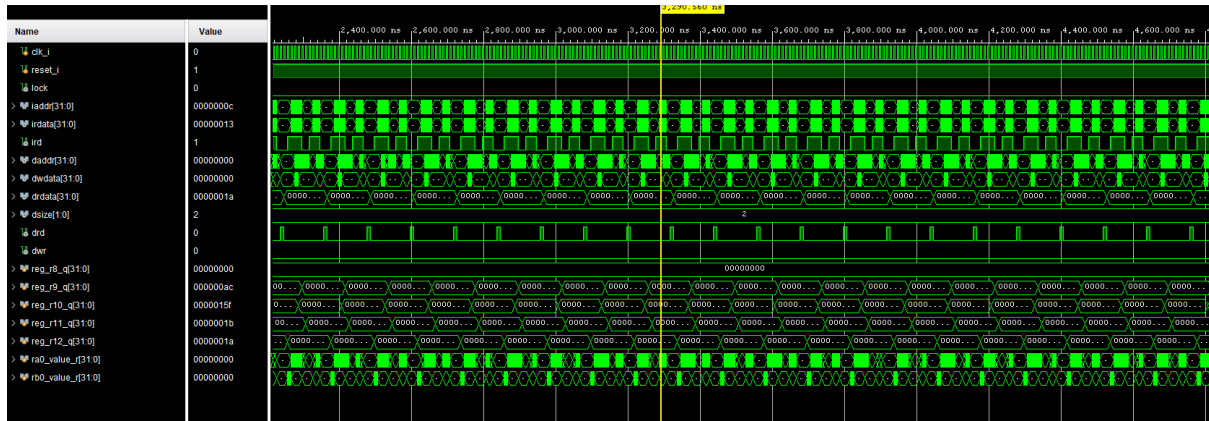
이를 통해 Baseline에서 사용한 Dummy 코드를 실제 Load 및 Store 기능으로 교체한 결과, ALU에서 계산된 메모리 주소를 이용한 데이터 메모리 접근과 Register File의 Write Back이 정상적으로 수행됨을 확인하였다. 또한 Load와 Store 명령어에 따라 Read Enable(drd)과 Write Enable(dwr) 신호가 올바르게 생성되어 RISC-V Core의 메모리 접근 기능이 정상적으로 구현되었음을 확인하였다.

이러한 과정을 통해 Load와 Store 명령어는 ALU에서 계산한 메모리 주소를 이용하여 데이터 메모리에 접근하며, Load는 메모리의 데이터를 Register File로 전달하고 Store는 Register File의 데이터를 메모리에 저장하는 방식으로 동작한다는 것을 이해하였다. 또한 Waveform을 통해 메모리 접근 신호와 Write Back 과정이 실제로 어떻게 수행되는지 확인할 수 있었다.

Day6. LAB2 – RISC-V Core Integration

(a) baseline의 시뮬레이션 결과는 아래 사진과 같다.





Problem 5(a)에서는 이전 실습에서 구현한 Program Counter, Branch, Load/Store 기능을 포함한 Baseline RISC-V Core를 이용하여 배열의 모든 원소를 더하는 프로그램을 실행하였다. 시뮬레이션 결과는 아래 그림과 같다.

Waveform을 보면 Program Counter(iaddr)는 반복문이 수행되는 동안 Branch 명령어에 의해 Loop의 시작 주소로 반복적으로 분기하며 프로그램이 정상적으로 실행되는 것을 확인할 수 있다. 또한 Load 명령어가 수행될 때마다 데이터 메모리의 주소가 $0x00000040 \rightarrow 0x00000044 \rightarrow 0x00000048 \rightarrow \dots$ 과 같이 4Byte씩 증가하는 것을 확인할 수 있으며, 이는 배열 A의 각 원소를 순차적으로 읽어오는 과정임을 알 수 있다. 읽어온 데이터는 Register x12에 저장된 후, ADD 명령어를 통해 누적 합(Register x10)에 더해지는 것을 확인하였다.

반복문이 수행되는 동안 Register x11은 반복 횟수를 나타내는 변수(i)로 사용되며 $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 64$ 까지 증가한다. Branch 조건이 참인 동안에는 Program Counter가 Loop의 시작 주소로 분기하여 다음 배열 원소를 계속 읽으며, x11이 64가 되면 Branch 조건이 거짓이 되어 반복문이 종료되는 것을 확인하였다.

최종적으로 Register x9(r9)의 값은 320($0x140$)이 되었다. 이는 프로그램 시작 시 x9가 배열의 시작 주소인 64($0x40$)로 초기화되고, 반복문이 수행될 때마다 `addi x9, x9, 4` 명령어에 의해 4Byte씩 증가하기 때문이다. 총 64번 반복하므로 최종 주소는 $64 + (64 \times 4) = 320$ 이 되며, 이는 배열의 마지막 원소 다음 주소를 가리키게 된다.

또한 Register x10(r10)의 최종 값은 2016($0x7E0$)이 되었다. 이는 배열 A에 저장된 0부터 63까지의 값을 모두 더한 결과이다. 배열의 합은

$$0 + 1 + 2 + \dots + 63 = \frac{63 \times 64}{2} = 2016$$

으로 계산되며, Waveform에서도 반복문이 수행될수록 x10의 값이 계속 증가하여 최종적으로 2016이 되는 것을 확인하였다.

이를 통해 Program Counter의 반복 분기, Load 명령어를 이용한 배열 접근, ALU를 이용한 누적 합 계산, 그리고 Branch 조건에 따른 반복문의 종료까지 RISC-V Core가 정상적으로 동작함을 확인하였다.

(b) Branch-Not-Equal (BNE)의 시뮬레이션 결과는 아래 사진과 같다.



Problem 5(b)에서는 기존 Branch-Less-Than(BLT) 명령어를 Branch-Not-Equal(BNE) 명령어로 변경하여 동작을 확인하였다. 이를 위해 기존 mem.hex에서 BLT 명령어를 BNE 명령어로 수정한 mem_bne.hex를 생성하였으며, Testbench 역시 새로운 Memory 파일을 사용할 수 있도록 riscv_core_bne_tb.v로 수정하였다. 또한 Branch Enable 생성 방식을 ALU에서 생성된 Flag를 이용하도록 구현하여, 두 Register의 값이 서로 다른 경우에만 Branch가 수행되도록 수정하였다. 시뮬레이션 결과는 아래 그림과 같다.

Waveform을 보면 Program Counter(iaddr)는 반복문이 수행되는 동안 0x0000000C → 0x00000010 → 0x00000014 → ... → 0x00000024의 실행 흐름을 반복하는 것을 확인할 수 있다. 이는 BNE 명령어가 수행될 때마다 Branch 조건이 만족되어 Loop의 시작 주소로 정상적으로 분기하고 있음을 의미한다. 또한 Load 명령어가 수행될 때마다 데이터 메모리 주소(daddr)는 0x00000040 → 0x00000044 → 0x00000048 → 0x0000004C → ...과 같이 4Byte씩 증가하며 배열의 각 원소를 순차적으로 읽어오는 것을 확인하였다.

Register File의 값 역시 정상적으로 변화하는 것을 확인하였다. Register x9(r9)는 배열의 주소를 저장하는 Pointer로 사용되며 반복문이 수행될 때마다 4Byte씩 증가하였다. 또한 Register x10(r10)은 배열 원소의 누적 합을 저장하며 Load된 데이터가 더해질 때마다 값이 증가하는 것을 확인하였다. 반복 횟수를 저장하는 Register x11(r11) 역시 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$ 과 같이 1씩 증가하며 반복문의 진행 상태를 나타내는 것을 확인하였다.

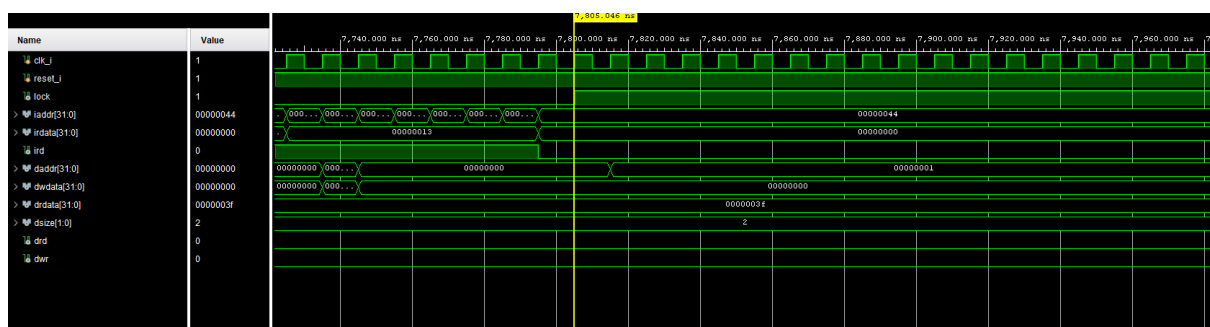
BNE 명령어의 Branch Enable은 ALU에서 생성한 Zero Flag를 이용하여 구현하였다. 두 Register의 값이 서로 같을 경우 Zero Flag가 1이 되므로 Branch는 수행되지 않고, 서로 다른 경우에는 Zero Flag가 0이 되어 Branch Enable이 활성화된다. 따라서 반복문이 수행되는 동안에는 Branch가 계속 수행되며, 반복 횟수(Register x11)가 종료 조건(Register x13)에 도달하면 Branch 조건이 거짓이 되어 반복문이 종료되는 것을 확인하였다.

이를 통해 기존 BLT 방식과 동일한 Branch Target Address 계산을 사용하면서도 Branch 조건만 BNE 방식으로 변경하여 정상적으로 동작함을 확인하였다. 또한 ALU에서 생성한 Flag를 이용하여 Branch Enable을 생성함으로써 Branch 판단이 정상적으로 수행되고, Program Counter와 Register File, Load 명령어가 모두 올바르게 동작하는 것을 확인하였다.

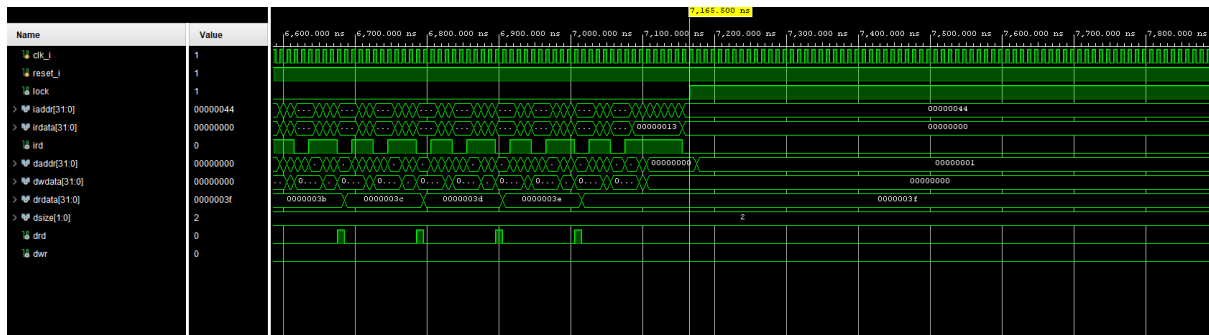
이번 실습을 통해 Load 및 Branch 명령어를 이용하여 배열을 순차적으로 탐색하고 누적 합을 계산하는 RISC-V Core의 전체 실행 과정을 이해할 수 있었다. 또한 Waveform을 통해 Register 값의 변화와 Program Counter의 반복 분기 과정을 직접 확인하면서 Core의 동작 흐름을 분석할 수 있었다. 특히 Branch 명령어는 Register 값을 직접 비교하는 것이 아니라 ALU에서 생성한 Flag를 이용하여 Branch 여부를 판단한다는 점을 이해하였으며, BNE 명령어를 구현하는 과정에서 Branch 조건만 변경하더라도 동일한 Branch Target Address 계산 방식을 사용할 수 있고, Program Counter와 반복문의 실행 흐름이 정상적으로 유지되는 것을 확인할 수 있었다.

Day6. LAB3 – Optimization

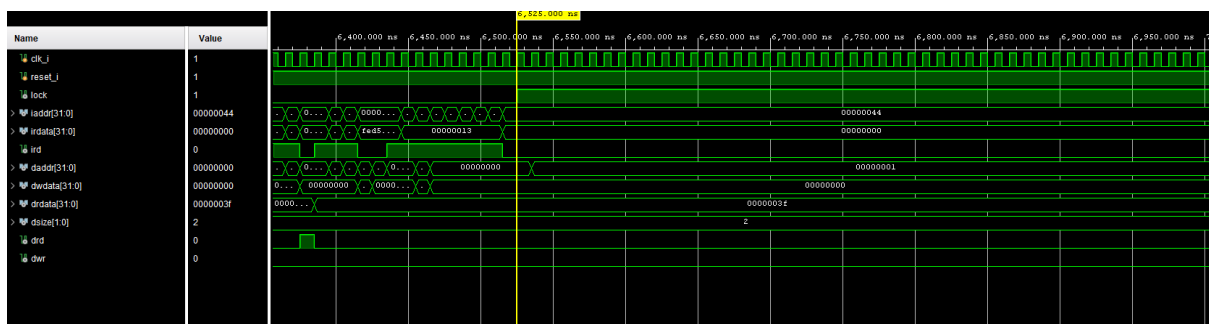
(1) Baseline – mem.hex



(2) mem_opt1.hex



(3) mem_opt2.hex



이번 실습에서는 동일한 기능을 수행하는 세 가지 Assembly 프로그램(Baseline, opt1, opt2)을 각각 실행하여 수행 시간을 비교하였다. 시뮬레이션 결과 Baseline은 7805 ns, opt1은 7165 ns, opt2는 6525 ns의 실행 시간을 나타냈으며, 명령어 재배치를 통해 프로그램의 수행 시간이 점차 감소하는 것을 확인하였다.

Baseline에서는 lw 명령어로 메모리에서 데이터를 읽은 직후 add 명령어가 바로 해당 데이터를 사용한다. 이 경우 add 명령어는 lw의 결과가 아직 Register File에 기록되기 전에 데이터를 필요로 하므로 RAW(Read After Write) Data Hazard가 발생한다. 일반적인 5단계 Pipeline(IF, ID, EX, MEM, WB)에서는 Load 명령어의 결과가 MEM과 WB 단계를 거친 후에야 사용할 수 있기 때문에, 다음 명령어가 이를 즉시 참조하면 Pipeline은 데이터를 기다리기 위해 Stall(Bubble)을 삽입하게 된다. 이러한 Stall은 프로그램의 실행 시간을 증가시키는 원인이 된다고 한다.

opt1에서는 lw와 add 사이에 주소를 증가시키는 독립적인 명령어(addi x9, x9, 4)를 배치하였다. 이 명령어는 lw의 결과를 사용하지 않으므로 Load 명령어가 완료되는 동안 독립적으로 수행될 수 있다. 따라서 add 명령어가 실행될 때에는 필요한 데이터가 준비되어 있어 Pipeline Stall이 감소하며, 실행 시간이 640 ns 단축되는 결과를 얻었다.

opt2에서는 lw와 add 사이에 주소 증가(addi x9, x9, 4)와 반복 변수 증가(addi x11, x11, 1)를 연속으로 배치하였다. 두 명령어 모두 lw의 결과와 의존성이 없으므로 Load 명령어가 완료될 충분한 시간을 확보할 수 있다. 그 결과 RAW Hazard로 인한 Stall이 더욱 감

소하여 실행 시간이 6525 ns로 줄어들었으며, Baseline 대비 1280 ns의 성능 향상을 확인하였다.

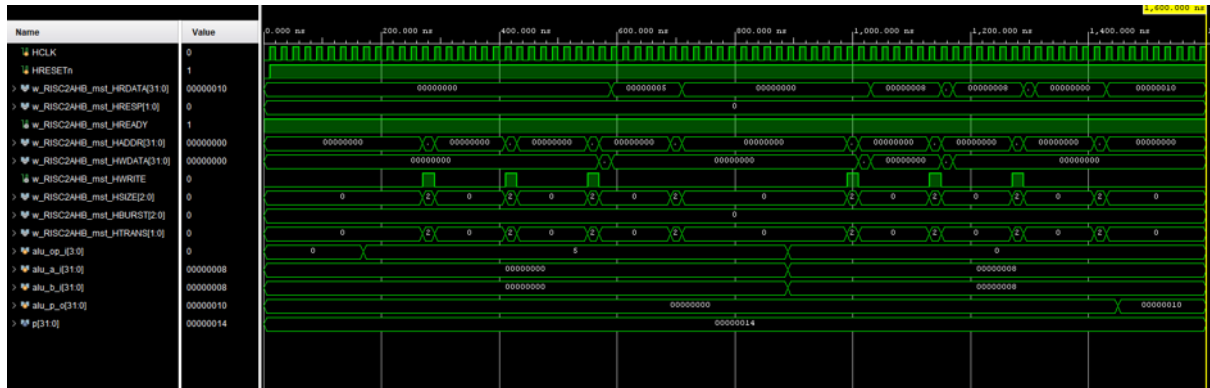
이번 실습에서 사용한 최적화 방법은 하드웨어를 변경하는 Data Forwarding, Register Renaming, Out-of-Order Execution과 같은 기법이 아니라, 컴파일 단계에서 독립적인 명령어를 재배치하는 Compiler Scheduling 기법이다. Compiler Scheduling은 프로그램의 동작 결과를 변경하지 않으면서 데이터 의존성이 없는 명령어를 Load 명령어와 의존 명령어 사이에 배치하여 Data Hazard를 완화하는 방법이다. 이러한 기법은 임베디드 시스템과 같은 비교적 단순한 프로세서에서도 적용할 수 있으며, 별도의 하드웨어 추가 없이 Pipeline Stall을 줄여 실행 성능을 향상시킬 수 있다.

실험 결과를 통해 동일한 프로그램이라도 명령어의 실행 순서만 적절히 재배치하면 Data Hazard를 효과적으로 감소시킬 수 있으며, Pipeline의 유휴 시간을 줄여 전체 실행 시간을 단축할 수 있음을 확인하였다. 또한 Compiler Scheduling은 별도의 하드웨어 변경 없이 성능을 향상시키는, 적절히 사용한다면 상당히 효과적인 방법이라는 점을 알게 되었다.

Day7. LAB1 – ALU IP

본 실습에서는 ALU IP를 AHB Bus에 연결하기 위해 AHB Slave Interface(riscv_alu_if.v)를 구현하였다. AHB Slave Interface는 AHB Master로부터 전달되는 Address, Write Data 및 Control 신호를 이용하여 ALU의 입력 레지스터를 제어하고, ALU에서 수행한 연산 결과를 AHB Bus를 통해 Master가 읽을 수 있도록 설계하였다. Register Map은 ALU Operation(0x00), Operand A(0x04), Operand B(0x08), ALU Result(0x0C)로 구성하였으며, ALU Result Register는 연산 결과만 읽을 수 있도록 Read Only Register로 구현하였다. 또한 Address Phase에서는 HADDR[3:2]를 이용하여 접근할 Register를 선택하고 HWRITE 신호를 저장하여 Read 또는 Write Transaction을 구분하도록 Address Decode Logic을 구현하였다. 이후 Data Phase에서는 선택된 Register에 따라 ALU Operation, Operand A 및 Operand B 레지스터에 데이터를 저장하는 Write Logic과, 선택된 Register의 값을 HRDATA로 출력하는 Read Logic을 구현하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 AHB Master가 각 Register에 대해 순차적으로 Write Transaction을 수행하는 것을 확인할 수 있다. 먼저 ALU Operation Register에 연산 코드가 저장되고, 이후 Operand A Register와 Operand B Register에 입력 데이터가 차례대로 기록된다. Write Transaction 동안 HWRITE 신호가 활성화되며, Address Phase에서 선택된 Register 번호(q_sel_sl_reg)에 따라 해당 Register만 갱신되는 것을 확인하였다. 입력 데이터가 모두 저장된 후 ALU는 지정된 Operation에 따라 연산을 수행하고, 계산 결과를 출력 포트 alu_p_o에 생성한다.

이후 Read Transaction에서는 ALU Result Register(0x0C)에 접근하여 연산 결과를 읽어 오는 과정이 정상적으로 수행되는 것을 확인하였다. Waveform에서 alu_a_i와 alu_b_i에는 각각 8(0x00000008)이 저장되었으며, ALU Operation은 ADD로 설정되었다. 이에 따라 ALU의 출력인 alu_p_o는 16(0x00000010)을 생성하였고, Read Logic을 통해 동일한 값이 HRDATA로 전달되는 것을 확인하였다.

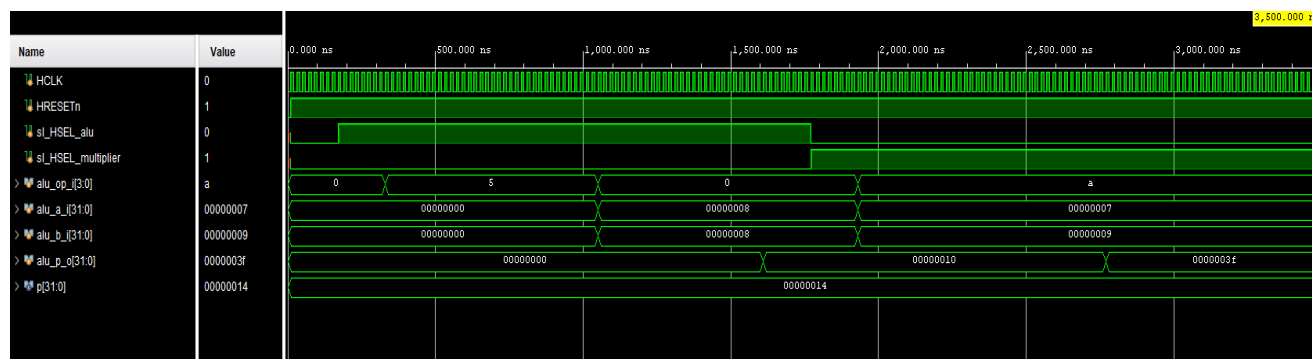
실험 결과, 구현한 AHB Slave Interface는 AHB Protocol의 Address Phase와 Data Phase를 정상적으로 처리하였으며, Register Mapping, Address Decode, Read Logic 및 Write Logic이 모두 올바르게 동작함을 확인하였다. 또한 AHB Master와 ALU IP 간의 Read/Write Transaction이 정상적으로 수행되는 것을 확인하고, AHB Bus를 통해 ALU를 제어하고 연산 결과를 읽어올 수 있음을 이해할 수 있었다.

Day7. LAB2 – Multi-AHB-Slave system

본 실습에서는 하나의 AHB Master와 두 개의 AHB Slave(ALU, Multiplier)로 구성된 Multi-AHB-Slave System을 구현하였다. Problem 7에서 구현한 riscv_alu_if.v를 그대로 재사용하였으며, riscv_multiplier_if.v의 AHB Slave Interface와 top_system.v의 Top Module을 완성하였다. Top Module에서는 AHB Master의 주소(Address)를 기반으로 ALU와 Multiplier 중 하나를 선택하도록 Address Decode Logic을 구현하였으며, 선택된 Slave의 HRDATA, HRESP, HREADY 신호를 Master로 전달하는 Multiplexer(MUX)를 설계하였다. 또

한 Multiplier는 Register Map을 이용하여 Operation, Operand A, Operand B, Signed Option 및 연산 결과를 AHB Bus를 통해 제어하고 읽어올 수 있도록 구현하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 초기에는 sl_HSEL_alu가 활성화되어 AHB Master가 ALU Slave에 접근하는 것을 확인할 수 있다. ALU Operation Register에는 연산 코드가 저장되고, Operand A와 Operand B Register에 입력 데이터가 순차적으로 기록된 후 ALU가 연산을 수행한다. 이후 sl_HSEL_multiplier가 활성화되면서 Master는 Multiplier Slave를 선택하여 동일한 방식으로 Register에 접근하고 데이터를 전달한다. 각 Slave는 선택된 경우에만 동작하며, 선택되지 않은 Slave는 Bus에 영향을 주지 않는 것을 확인하였다.

Waveform에서 ALU 동작 시 alu_op_i는 ADD(0x5)로 설정되었으며, alu_a_i와 alu_b_i에는 각각 8(0x00000008)이 저장되어 alu_p_o는 16(0x00000010)을 출력하였다. 이후 Multiplier가 선택된 후에는 alu_op_i가 0xA로 변경되고, 입력값으로 7(0x00000007)과 9(0x00000009)가 전달되었다. 이에 따라 Multiplier의 연산 결과는 63(0x0000003F)으로 생성되었으며, 선택된 Slave의 출력이 AHB Bus를 통해 정상적으로 Master로 전달되는 것을 확인하였다.

실험 결과, 구현한 Multi-AHB-Slave System은 AHB Master가 주소에 따라 ALU와 Multiplier를 정상적으로 선택하여 접근할 수 있음을 확인하였다. 또한 Address Decode Logic과 Slave Selection Logic, Register Mapping 및 Read/Write Logic이 모두 정상적으로 동작하였으며, 하나의 AHB Bus를 통해 두 개의 Slave IP를 안정적으로 제어하고 연산 결과를 읽어올 수 있음을 검증하였으며, 하나의 AHB Master가 여러 개의 Slave를 선택하여 제어하는 방법을 이해할 수 있었다.

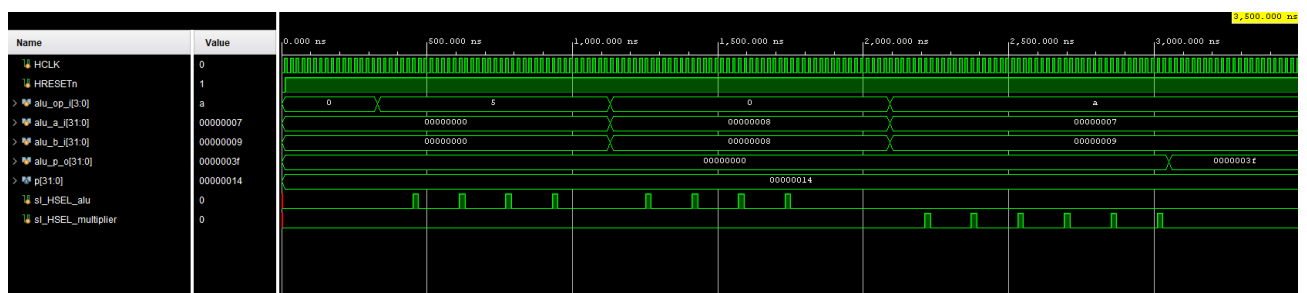
Day7. LAB3 – Multiplier

본 실습에서는 하나의 AHB Master와 두 개의 AHB Slave(ALU, Multiplier)로 구성된 Multi-AHB-Slave System을 구현하였다. Problem 7과 Problem 8에서 구현한 riscv_alu_if.v

와 riscv_multiplier_if.v를 재사용하였으며, top_system.v에서 AHB Decoder(Address Decode Logic)와 Slave Selection Logic을 구현하였다. Address Decoder는 AHB Master가 전송하는 주소를 기반으로 ALU와 Multiplier 중 하나의 Slave를 선택하도록 설계하였으며, 선택된 Slave의 HRDATA, HRESP, HREADY 신호를 Multiplexer(MUX)를 통해 Master로 전달하도록 구현하였다. 또한 Multiplier는 Register Map을 이용하여 Operation Code, Operand A, Operand B, Signed Option, 그리고 연산 결과를 AHB Bus를 통해 읽고 쓸 수 있도록 구현하였다.

초기 시뮬레이션에서는 Multiplier의 입력값이 정상적으로 전달되지 않아 연산 결과(alu_p_o)가 계속 0으로 출력되는 문제가 발생하였다. 원인을 분석한 결과, AHB Master Testbench에서 Address 및 Data 신호가 Clock Edge와 동시에 변경되면서 Race Condition이 발생하였고, Slave Interface가 데이터가 갱신되기 이전의 값을 샘플링하는 것을 확인하였다. 이를 해결하기 위해 ahb_master.v의 Test Task에서 Propagation Delay(#p)를 활성화하여 Address 및 Data 신호가 Clock 이후 안정적으로 전달되도록 수정하였다. 수정 이후에는 Multiplier Interface가 Operand를 정상적으로 저장하고 연산 결과를 올바르게 출력하는 것을 확인하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 초기에는 sl_HSEL_alu가 활성화되어 AHB Master가 ALU Slave를 선택하는 것을 확인할 수 있다. 먼저 ALU Operation Register에 연산 코드가 기록되고, 이후 Operand A와 Operand B Register에 입력값이 순차적으로 저장된다. 첫 번째 연산에서는 ALU_SLT(0x5) 명령이 수행되며 입력값은 0과 0으로 설정되어 결과값은 0이 출력된다. 이후 두 번째 연산에서는 ALU_ADD(0x0) 명령이 수행되고 Operand A와 Operand B에 각각 8이 저장되어 16(0x00000010)의 결과가 생성되는 것을 확인할 수 있다.

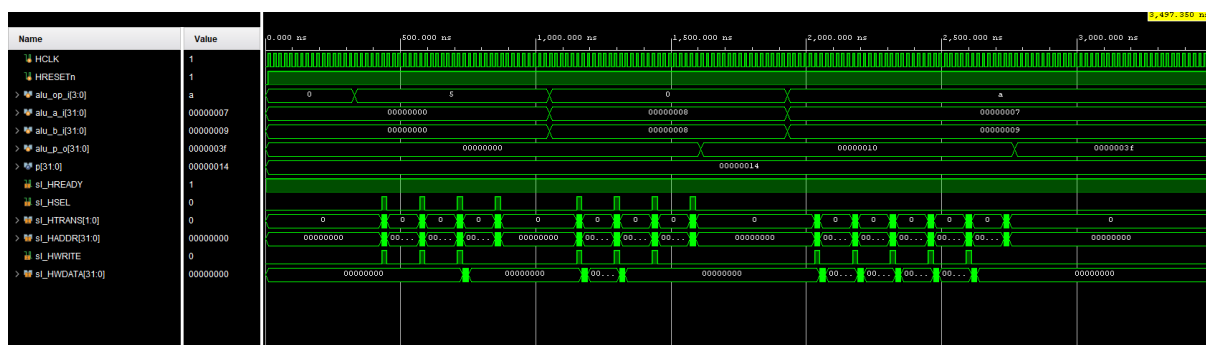
이후 sl_HSEL_multiplier가 활성화되면서 AHB Master는 Multiplier Slave를 선택한다. Multiplier Operation Register에는 ALU_MULL(0xA)가 기록되고 Operand A와 Operand B에는 각각 7과 9가 저장된다. Propagation Delay를 적용한 이후에는 Operand가 정상적으로 Multiplier 내부 레지스터에 저장되며, 곱셈 결과인 63(0x0000003F)이 생성되어 alu_p_o를 통해 출력되는 것을 확인하였다. 또한 선택된 Slave만이 Bus를 구동하며, 선택되지 않은 Slave는 Bus에 영향을 주지 않는 것을 확인하였다.

실험 결과, 구현한 AHB Decoder는 AHB Master의 주소를 기반으로 ALU와 Multiplier Slave를 정상적으로 선택하였으며, Address Decode Logic과 Slave Selection Logic, Register Mapping 및 Read/Write Logic이 모두 정상적으로 동작함을 확인하였다. 또한 Testbench에서 발생한 Race Condition을 Propagation Delay를 이용하여 해결함으로써 AHB Protocol의 Address Phase와 Data Phase가 안정적으로 동작함을 검증하였다. 이를 통해 하나의 AHB Master가 여러 개의 Slave IP를 주소 기반으로 선택하고 제어하는 방법과, 시뮬레이션 환경에서 Timing 및 Propagation Delay가 올바른 데이터 전달에 중요한 요소임을 확인할 수 있었다.

Day7. LAB4 – AHB Bus Interconnection

본 실습에서는 AHB Bus를 이용한 Top System을 구현하였다. 하나의 AHB Master(RISC-V Dummy)와 두 개의 AHB Slave(ALU, Multiplier)를 하나의 AHB Bus에 연결하였으며, 기존에 구현한 riscv_alu_if.v와 riscv_multiplier_if.v를 그대로 재사용하여 top_system.v를 완성하였다. Top Module에서는 AHB Master와 Slave 사이의 Bus Interconnection을 구현하였으며, Master가 출력하는 Address(HADDR), Write Data(HWDATA) 및 제어 신호(HWRITE, HTRANS, HSIZE, HBURST)를 각 Slave로 전달하도록 구성하였다. 또한 Address Decode Logic을 이용하여 접근 주소에 따라 ALU 또는 Multiplier를 선택하도록 구현하였으며, 선택된 Slave의 Read Data(HRDATA), Response(HRESP), Ready(HREADY) 신호를 Master로 전달하는 Read Data Multiplexer를 설계하였다. 이를 통해 하나의 AHB Bus에서 여러 Slave IP를 공유하여 사용할 수 있는 구조를 구현하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 AHB Master가 먼저 ALU 영역의 주소를 접근하여 Operand A, Operand B 및 Operation Register에 데이터를 순차적으로 기록한 후 연산 결과를 읽어오는 것을 확인할 수 있다. 이후 Address Decode Logic에 의해 Multiplier 영역이 선택되면서 동일한 방식으로 Operand와 Operation을 전달하고 연산 결과를 읽어온다. 이 과정에

서 sl_HSEL 신호가 Address에 따라 활성화되며, 선택된 Slave만 AHB Bus에 응답하고 선택되지 않은 Slave는 Bus에 영향을 주지 않는 것을 확인하였다.

Waveform에서 초기에는 ALU의 SLT 연산과 ADD 연산이 수행된다. 먼저 alu_op_i는 SLT(0x5) 로 설정되고 Operand A와 Operand B는 모두 0이 입력되어 결과값은 0이 출력된다. 이후 alu_op_i가 ADD(0x0) 로 변경되고 Operand A와 Operand B에 각각 8(0x00000008) 이 저장되어 alu_p_o는 16(0x00000010) 을 출력하는 것을 확인하였다. 이후 Master는 Multiplier 영역으로 접근하여 alu_op_i를 MULL(0xA) 로 설정하고 Operand A와 Operand B에 각각 7(0x00000007) 과 9(0x00000009) 를 기록하였다. Multiplier는 두 값을 곱하여 63(0x0000003F) 을 생성하였으며, Read Transaction을 통해 해당 결과가 정상적으로 Master로 전달되는 것을 확인하였다. 또한 sl_HREADY는 항상 High 상태를 유지하여 Wait State 없이 모든 AHB Transaction이 정상적으로 완료되는 것을 확인하였다.

실험 결과, 구현한 AHB Bus Interconnection은 하나의 AHB Master가 Address Decode Logic을 통해 여러 Slave를 정상적으로 선택하여 접근할 수 있음을 확인하였다. 또한 Address Bus, Control Bus, Write Data Bus 및 Read Data Multiplexer가 정상적으로 동작하여 ALU와 Multiplier를 하나의 Bus에서 안정적으로 제어할 수 있음을 검증하였다. 특히 Address에 따라 Slave를 자동으로 선택하고, 선택된 Slave만 Bus에 응답하도록 구성함으로써 AHB Bus 기반의 Bus Interconnection 구조와 Top Module 설계 방법을 이해할 수 있었다.